

Your First Local AI Chat App

A gentle, practical guide to connecting a browser interface to a Python backend and an AI model.

1 • Browser HTML, CSS and JavaScript	2 • Backend FastAPI validates and routes	3 • Model Ollama or a hosted provider replies
Collects a message and shows the answer.	Keeps provider settings private; stores chat history.	Receives only server-to-server API requests.

What you will build

A small, locally run learning app. The browser sends a message to `/api/chat`; Python adds the current browser's saved conversation and calls the configured provider; the reply comes back to the browser. MySQL stores chats separately using a private, HttpOnly browser cookie.

Project map

File / folder	Purpose
<code>app.py</code>	FastAPI backend, API routes, cookie session and MySQL storage.
<code>practice.py</code>	OpenAI-compatible client; connects to Ollama, OpenAI, or OpenRouter.
<code>frontend/</code>	Browser interface: HTML, CSS and JavaScript.
<code>.env.local</code>	Private provider key and local settings; do not commit.
MySQL database	Local MySQL server stores chat messages.
<code>requirements.txt</code>	Python packages needed by the app and tests.

1. Set it up on Windows

Open PowerShell in the project folder. Create an isolated Python environment, activate it, and install the project packages:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

If PowerShell blocks activation, use `.\.venv\Scripts\python.exe` in place of `python` in the commands below.

Install Ollama and configure the app

Install Ollama for Windows from ollama.com/download/windows. After installation, Ollama runs locally; open PowerShell and download the Llama 3.2 model used by this app. Its first download can take time and disk space.

```
ollama pull llama3.2:3b
ollama run llama3.2:3b
```

The second command starts a simple terminal chat; type `/bye` to leave. Copy `.env.example` to `.env.local` for a new setup; if `.env.local` already exists, preserve its MySQL values and update only the LLM settings. The app leaves any existing `.env` file untouched. Your local MySQL service must also be running.

```
LLM_PROVIDER=ollama
LLM_MODEL=llama3.2:3b
LLM_BASE_URL=http://127.0.0.1:11434/v1
LLM_DEMO_MODE=false
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_DATABASE=genai_chat_practice
MYSQL_USER=root
MYSQL_PASSWORD=your-local-mysql-password
COOKIE_SECURE=false
```

Ollama runs the model on your computer and does not need a cloud API key or paid API account. LM Studio is another local option: set `LLM_PROVIDER=lmstudio`, `LLM_BASE_URL=http://127.0.0.1:1234/v1`, and `LLM_MODEL` to the model ID loaded in LM Studio, then start its Local Server. For hosted alternatives, set `LLM_PROVIDER=openai` and `OPENAI_API_KEY`, or use `LLM_PROVIDER=openrouter` and `OPENROUTER_API_KEY`. Hosted usage may cost money; check provider pricing and limits. OpenRouter free-model availability can change. Demo mode is optional: set `LLM_DEMO_MODE=true` to return sample echoes without making any model request. Never put real keys or passwords in JavaScript, HTML, a screenshot, or Git. The `.env.local` file is ignored by Git. If using OpenAI, create a key at platform.openai.com/api-keys, then check API billing at the [OpenAI billing page](#). API usage is billed separately from a ChatGPT subscription.

Start the backend and frontend

```
python -m uvicorn app:app --reload --host 127.0.0.1 --port 8000
```

Open **`http://127.0.0.1:8000`** in your browser. FastAPI serves the frontend and backend from the same local server, so no separate frontend server or CORS setup is needed. Press Ctrl+C in PowerShell to stop.

Try it and run the tests

```
python -m unittest -v test_app
```

The tests use a fake model reply and a temporary MySQL database: they check routing, history, and browser session separation without hosted API requests or quota. `http://127.0.0.1:8000/docs` shows the backend API.

2. How the browser connects to Python

1	The browser loads the page from FastAPI at /.
2	frontend/app.js sends JSON to POST /api/chat.
3	FastAPI validates the message and reads the browser's session history from MySQL.
4	practice.py sends the message list to the selected OpenAI-compatible provider.
5	The backend saves the user message and reply, then returns JSON to the browser.
6	The browser renders the reply as text (not executable HTML).

Useful backend endpoints

Method + path	What it does
GET /api/health	Shows the server, model, and whether it is in demo or live mode.
GET /api/history	Returns the current browser session's saved messages.
POST /api/chat	Validates a message, asks the configured provider, and saves a successful turn.
POST /api/clear	Clears only the current browser session's chat.

The frontend and backend are same-origin: JavaScript calls relative paths such as `/api/chat`. The browser sends an `HttpOnly` session cookie automatically. JavaScript never receives the provider secret or a caller-chosen database session ID.

```
Browser -- POST /api/chat {"message": "Hi"} --> FastAPI
FastAPI -- server-side model request --> Ollama / hosted provider
Browser <-- {"reply": "Hello!"} ----- FastAPI
```

3. Chat memory, errors, and safe habits

How follow-up questions work

The model does not automatically know what you asked on an earlier request. This app loads up to the last 20 saved messages for the current browser session, adds the new question, and sends that list together. A successful user/assistant pair is saved to MySQL. Each browser gets a random, HttpOnly session cookie; /clear deletes only that session's rows. Restarting the server does not delete MySQL history.

If something goes wrong

What you see	What to check
Replies only echo your prompt	Demo mode is on; set LLM_DEMO_MODE=false and restart.
Cannot reach Ollama	Install/start Ollama, run ollama pull llama3.2:3b, then retry.
Hosted provider key is missing	Set OPENAI_API_KEY or OPENROUTER_API_KEY in .env.local.
Provider rejected the key	Check the key, account access, and provider settings.
Rate or usage limit reached	Check the selected provider's usage limits and billing.
Could not connect to a hosted provider	Check internet access and provider settings.
Page does not load	Confirm Uvicorn is still running and visit http://127.0.0.1:8000.

Learning code, not public deployment

This is a local practice app, not a ready-to-publish multi-user service. Before putting it on the public internet, add user authentication and authorization, HTTPS, rate limiting, CSRF protection, secure secret storage, privacy/retention controls, monitoring, and a production database. Use COOKIE_SECURE=true only behind HTTPS. Keep Uvicorn bound to 127.0.0.1 while learning.

A good learning sequence

Recommended no-key first test: Ollama with llama3.2:3b. For hosted models, check current availability and price before use. 1) Send one message. 2) Read the JSON reply. 3) Ask a follow-up. 4) Inspect the browser Network panel and the /api/chat request. 5) Find the matching route in app.py and the provider call in practice.py. 6) Change LLM_MODEL or LLM_PROVIDER and observe what happens.